

COMP47670 Data Science in Python - Lab Tasks Question Bank + Solutions (Study Notes)

Organised into Exam-Style Task 1 / Task 2 / Task 3. Generated from your provided lab solution notebooks.

Task 1 - Data Loading, Cleaning, EDA, Visualisation

This section contains lab questions and solutions that train the core workflow: load data, clean, summarise, aggregate, and visualise.

Lab 01 Tasks - Solutions

Setup / Imports used in this lab:

```
english = ["hello", "cat", "dog", "yes", "tomorrow", "yesterday", "fish",
           "difficult", "easy", "bad", "no", "tuesday", "january", "world"]
french = ["bonjour", "chat", "chien", "oui", "demain", "hier", "poisson",
          "difficile", "facile", "mauvais", "non", "mardi", "janvier", "monde"]
```

Task 1

Task 1

Verify that: 1) both lists contain an equal number of values; 2) all elements in both lists are strings.

(Hint: We can use the `isinstance()` function to check whether a value has a specified type)

Solution code:

```
# check if the lists have the same length
if len(english) == len(french):
    print("Both lists have the same number of values")
else:
    print("The lists have different lengths")

# check first list contains only strings
for value in english:
    if not isinstance(value, str):
        print("Not a valid string:", value)

# check second list contains only strings
for value in french:
    if not isinstance(value, str):
        print("Not a valid string:", value)
```

What to change for a different dataset:

- Change the input file/URL (e.g., triathlon.csv, olympics2012.csv) to your dataset path.
- Update column names used in calculations (e.g., Gold/Silver/Bronze, Swim/Cycle/Run, Date/Close).
- If the dataset uses different missing-value markers, adjust cleaning (dropna vs fillna).
- If you group by a different category (e.g., Region instead of Province), update the groupby/crosstab columns.
- For plots, update the x/y columns and chart titles/labels.

Task 2

Task 2

Create a Python dictionary, which maps English words to French words, using the lists above.

Solution code:

```
# create an empty dictionary to store English-to-French word pairs
translator = {}
# loop through the English list with index using enumerate
for i, word in enumerate(english):
    # add the English word as the key and the matching French word as the value
    translator[word] = french[i]
translator
translator["hello"]
translator["tuesday"]
```

What to change for a different dataset:

- Change the input file/URL (e.g., triathlon.csv, olympics2012.csv) to your dataset path.
- Update column names used in calculations (e.g., Gold/Silver/Bronze, Swim/Cycle/Run, Date/Close).
- If the dataset uses different missing-value markers, adjust cleaning (dropna vs fillna).
- If you group by a different category (e.g., Region instead of Province), update the groupby/crosstab columns.
- For plots, update the x/y columns and chart titles/labels.

Task 3

Task 3

Remove all key-value pairs from the dictionary above, where the keys (i.e. the English words) contain less than 4 characters.

Solution code:

```
# create a list of words to remove (avoid modifying dict during iteration)
words_to_remove = []
for word in translator.keys():
    if len(word) < 4:
        words_to_remove.append(word)

# remove the words
for word in words_to_remove:
    del translator[word]
translator
```

What to change for a different dataset:

- Change the input file/URL (e.g., triathlon.csv, olympics2012.csv) to your dataset path.
- Update column names used in calculations (e.g., Gold/Silver/Bronze, Swim/Cycle/Run, Date/Close).
- If the dataset uses different missing-value markers, adjust cleaning (dropna vs fillna).

- If you group by a different category (e.g., Region instead of Province), update the groupby/crosstab columns.
- For plots, update the x/y columns and chart titles/labels.

Task 4

Task 4

For the remaining words, print an alphabetically-ordered list of the English words and their corresponding French translations.

Solution code:

```
# iterate over a sorted list of English words (dictionary keys)
for word in sorted(translator.keys()):
    # print the English word and its French translation
    print("%s -> %s" % (word, translator[word]))
```

What to change for a different dataset:

- Change the input file/URL (e.g., triathlon.csv, olympics2012.csv) to your dataset path.
- Update column names used in calculations (e.g., Gold/Silver/Bronze, Swim/Cycle/Run, Date/Close).
- If the dataset uses different missing-value markers, adjust cleaning (dropna vs fillna).
- If you group by a different category (e.g., Region instead of Province), update the groupby/crosstab columns.
- For plots, update the x/y columns and chart titles/labels.

Task 5

Task 5

Write a new function named `translate()` which accepts 3 arguments: a translation dictionary, a word to translate, and a default word. If the word is in the dictionary, the function should return the translated word. Otherwise, it should return the default word.

Solution code:

```
def translate(td, word, default_word):
    # return the translation if the word exists in the dictionary
    if word in td:
        return td[word]
    # otherwise, return the default word
    return default_word
translate(translator, "hello", "unknown")
translate(translator, "world", "unknown")
translate(translator, "december", "unknown")
```

What to change for a different dataset:

- Change the input file/URL (e.g., triathlon.csv, olympics2012.csv) to your dataset path.
- Update column names used in calculations (e.g., Gold/Silver/Bronze, Swim/Cycle/Run, Date/Close).

- If the dataset uses different missing-value markers, adjust cleaning (dropna vs fillna).
- If you group by a different category (e.g., Region instead of Province), update the groupby/crosstab columns.
- For plots, update the x/y columns and chart titles/labels.

Task 6

Task 6

Write a new function named `reverse_translate()` which accepts 2 arguments: a translation dictionary and a word in the target translation language. If the translation is found, it returns the original word; otherwise, it returns a default message indicating that the translation is not available

Solution code:

```
def reverse_translate(td, translated_word):
    # check if the translated word exists as a value in the dictionary
    for original_word, translation in td.items():
        if translation == translated_word:
            return original_word
    # if not found, return a default message
    return "not available"
reverse_translate(translator, "demain")
reverse_translate(translator, "bonjour")
reverse_translate(translator, "ville")
```

What to change for a different dataset:

- Change the input file/URL (e.g., triathlon.csv, olympics2012.csv) to your dataset path.
- Update column names used in calculations (e.g., Gold/Silver/Bronze, Swim/Cycle/Run, Date/Close).
- If the dataset uses different missing-value markers, adjust cleaning (dropna vs fillna).
- If you group by a different category (e.g., Region instead of Province), update the groupby/crosstab columns.
- For plots, update the x/y columns and chart titles/labels.

Lab 02 Tasks - Solutions

Task 1

Task 1

Create a function `sorted_random()` which generates a list of `*n*` random numbers, sorted in descending order (i.e. largest first). The `number *n*` should be an input argument for the function. If no value is specified for `*n*`, by default a list of 10 numbers should be generated.

Solution code:

```
# first we need to import the random number generation module
import random
# define our new function
```

```
def sorted_random(n=10):
    numbers = []
    for i in range(n):
        # generate the next random number and add it to our list
        numbers.append(random.random())
    # sort the numbers in descending order
    numbers.sort(reverse=True)
    return numbers
# test our function
print(sorted_random(2))
print(sorted_random(4))
# test with the default argument value
print(sorted_random())
```

What to change for a different dataset:

- Change the input file/URL (e.g., triathlon.csv, olympics2012.csv) to your dataset path.
- Update column names used in calculations (e.g., Gold/Silver/Bronze, Swim/Cycle/Run, Date/Close).
- If the dataset uses different missing-value markers, adjust cleaning (dropna vs fillna).
- If you group by a different category (e.g., Region instead of Province), update the groupby/crosstab columns.
- For plots, update the x/y columns and chart titles/labels.

Task 2

Task 2

Download the comma-separated file `*scores.csv*` from the module Brightspace page and save it to the same directory as your notebook.

Write a function `display_numbers()` which takes one argument - an input file path. The function should read floating point numbers from each line in the specified file, and compute the sum for the values on each line. Print each sum to 2 decimal places.

Apply the function `display_numbers()` to `*scores.csv*`.

Here's an improved version that includes proper error handling:

Solution code:

```
def display_numbers(in_path):
    # open the file and process each line
    with open(in_path, "r") as fin:
        for line in fin:
            line = line.strip()
            parts = line.split(",")
            values = []
            # convert each part to a float
            for s in parts:
                values.append(float(s))
            # print the sum of the values with 2 decimal places
```

```

        print(f"{sum(values):.2f}")
display_numbers("scores.csv")
def display_numbers_enhanced(in_path):
    try:
        with open(in_path, "r") as fin:
            for line_num, line in enumerate(fin, 1):
                try:
                    # Convert each value to float and calculate sum
                    values = [float(x) for x in line.strip().split(",")]
                    total = sum(values)
                    print(f"{total:.2f}")
                except ValueError as e:
                    print(f"Error on line {line_num}: Invalid number format -
{e}")
            except Exception as e:
                print(f"Error processing line {line_num}: {e}")
    except FileNotFoundError:
        print(f"Error: File '{in_path}' not found")
    except PermissionError:
        print(f"Error: Permission denied accessing '{in_path}'")
    except Exception as e:
        print(f"Unexpected error: {e}")
display_numbers_enhanced("scores.csv")

```

What to change for a different dataset:

- Change the input file/URL (e.g., triathlon.csv, olympics2012.csv) to your dataset path.
- Update column names used in calculations (e.g., Gold/Silver/Bronze, Swim/Cycle/Run, Date/Close).
- If the dataset uses different missing-value markers, adjust cleaning (dropna vs fillna).
- If you group by a different category (e.g., Region instead of Province), update the groupby/crosstab columns.
- For plots, update the x/y columns and chart titles/labels.

Task 3

Task 3

Write a function `reverse\_numbers()` which takes two arguments - 1) an input file path; 2) an output file path. The function should read floating point numbers from each line in the specified input file. The order of the values from in line should then be reversed, and these lines should be written to the specified output file.

Apply the function `reverse\_numbers()` to the file *scores.csv* from Task 2 above to create a new output file *reversed.csv*.

Here's an improved version of the function that includes proper error handling:

Solution code:

```

def reverse_numbers(in_path, out_path):
    # open input and output files

```

```

with open(in_path, "r") as fin, open(out_path, "w") as fout:
    for line in fin:
        # read and clean each line
        line = line.strip()
        parts = line.split(",")
        # reverse the list
        parts.reverse()
        # write the reversed values, separated by commas
        fout.write(",".join(parts))
        # add newline character
        fout.write("\n")

print(f"Successfully reversed numbers from '{in_path}' to '{out_path}'")
reverse_numbers("scores.csv", "reversed.csv")
def reverse_numbers_enhanced(in_path, out_path):
    try:
        with open(in_path, "r") as fin:
            with open(out_path, "w") as fout:
                for l in fin.readlines():
                    l = l.strip()
                    parts = l.split(",")
                    parts.reverse()
                    fout.write(",".join(parts))
                    fout.write("\n")
        print(f"Successfully reversed numbers from '{in_path}' to '{out_path}'")
    except FileNotFoundError:
        print(f"Error: One of the files was not found. Please check the paths: '{in_path}' or '{out_path}'")
    except IOError as e:
        print(f"Error: An I/O error occurred during file operation: {e}")
    except Exception as e:
        print(f"An unexpected error occurred: {e}")
reverse_numbers_enhanced("scores.csv", "reversed2.csv")

```

What to change for a different dataset:

- Change the input file/URL (e.g., triathlon.csv, olympics2012.csv) to your dataset path.
- Update column names used in calculations (e.g., Gold/Silver/Bronze, Swim/Cycle/Run, Date/Close).
- If the dataset uses different missing-value markers, adjust cleaning (dropna vs fillna).
- If you group by a different category (e.g., Region instead of Province), update the groupby/crosstab columns.
- For plots, update the x/y columns and chart titles/labels.

Task 4

Task 4

Download the comma-separated file *worldcup.csv* from the module Brightspace page and save it to the same directory as your notebook. This data file contains a list of the winners and runners-up for all FIFA World Cup finals, one final per line.

Write a function `count_finalists()` which takes a single argument (an input file path), reads the data, and displays the number of times each country has appeared in a World Cup final. Countries should be displayed in alphabetical order.

Apply the function `count_finalists()` to the file `*worldcup.csv*`.

Note that, if we wanted to sort the countries in descending order by the number of times that they have appeared in a final, we could use a Python `Counter` from the `collections` module, and call its associated `most_common()` function as shown below:

Solution code:

```
def count_finalists(in_path):
    # read the data from the file
    with open(in_path, "r") as fin:
        lines = fin.readlines()
    # skip the header line and process the rest
    counts = {}
    for line in lines[1:]:
        parts = line.strip().split(",")
        winner = parts[1]
        runner_up = parts[2]
        if not winner in counts:
            counts[winner] = 0
        counts[winner] += 1
        if not runner_up in counts:
            counts[runner_up] = 0
        counts[runner_up] += 1
    # sort country names alphabetically
    countries = sorted(counts.keys())
    for country in countries:
        print(f"{counts[country]}\t{country}")
count_finalists("worldcup.csv")
from collections import Counter

def count_finalists_v2(in_path):
    # read the data from the file
    with open(in_path, "r") as fin:
        lines = fin.readlines()
    # skip the header line and process the rest
    counts = Counter()
    for line in lines[1:]:
        parts = line.strip().split(",")
        winner = parts[1]
        runner_up = parts[2]
        counts[winner] += 1
        counts[runner_up] += 1
    # sort countries by their corresponding count
    for country, count in counts.most_common():
        print(f"{count}\t{country}")
count_finalists_v2("worldcup.csv")
```

What to change for a different dataset:

- Change the input file/URL (e.g., triathlon.csv, olympics2012.csv) to your dataset path.
- Update column names used in calculations (e.g., Gold/Silver/Bronze, Swim/Cycle/Run, Date/Close).
- If the dataset uses different missing-value markers, adjust cleaning (dropna vs fillna).
- If you group by a different category (e.g., Region instead of Province), update the groupby/crosstab columns.
- For plots, update the x/y columns and chart titles/labels.

Task 5

Task 5

Write a function `validate_csv_data()` which takes a file path as input and validates that each line contains only numeric values (floats or integers). The function should:

1. Read the file line by line
2. Check if each value in each line can be converted to a float
3. Print any validation errors with the line number
4. Return the total number of valid lines processed

Test your function with the `*scores.csv*` file and also create a test file with some invalid data to show that the validation process works correctly.

Solution code:

```
def validate_csv_data(file_path):
    valid_lines = 0
    try:
        # read the file
        with open(file_path, "r") as fin:
            for line_num, line in enumerate(fin, 1):
                line = line.strip()
                if not line: # skip empty lines
                    continue
                try:
                    values = line.split(",")
                    # try to convert each value to float
                    for i, value in enumerate(values):
                        float(value.strip())
                    # if we get here, all values are valid
                    valid_lines += 1
                    print(f"Line {line_num}: Valid ({len(values)} values)")
                except ValueError as e:
                    print(f"Line {line_num}: Invalid data - '{value.strip()}'
is not a number")
            except Exception as e:
                print(f"Line {line_num}: Error processing line - {e}")

    except FileNotFoundError:
```

```

        print(f"Error: File '{file_path}' not found")
        return 0
    except Exception as e:
        print(f"Error reading file: {e}")
        return 0

    return valid_lines
# first, create a test file with some invalid data
test_data = """85.5,92.0,78.5
90,invalid,87
95.5,89.0,91.5
,85,90
abc,def,ghi
88.0,92.5,85.0"""

with open("test_data.csv", "w") as f:
    f.write(test_data)
# now try out the function on the original file
print("Testing with valid data (scores.csv):")
res1 = validate_csv_data("scores.csv")
print(f"Found {res1} valid lines")
# now try out function on the new invalid file
print("Testing with invalid data (test_data.csv):")
res2 = validate_csv_data("test_data.csv")
print(f"Found {res2} valid lines")

```

What to change for a different dataset:

- Change the input file/URL (e.g., triathlon.csv, olympics2012.csv) to your dataset path.
- Update column names used in calculations (e.g., Gold/Silver/Bronze, Swim/Cycle/Run, Date/Close).
- If the dataset uses different missing-value markers, adjust cleaning (dropna vs fillna).
- If you group by a different category (e.g., Region instead of Province), update the groupby/crosstab columns.
- For plots, update the x/y columns and chart titles/labels.

Lab 04 Tasks - Solutions

Task 1

Task 1 - JSON Parsing

Use Python to download a file containing metadata for member states of the EU in JSON format from the URL:

<http://mlg.ucd.ie/modules/python/eu.json>

Parse the JSON data that you have downloaded.

From the parsed JSON data, print a list of all of the country names, along with the corresponding capital city.

Extract the population information for each member state.

Export the EU member state data to a new file in CSV format, with an appropriate header line.

Solution code:

```
import urllib.request
import urllib.error

url = "http://mlg.ucd.ie/modules/python/eu.json"

try:
    # fetch the specified URL
    response = urllib.request.urlopen(url)
    raw_json = response.read().decode("utf-8")
    print("Successfully downloaded EU data")
    print(raw_json)
except urllib.error.HTTPError as e:
    print(f"HTTP Error {e.code}: {e.reason}")
except urllib.error.URLError as e:
    print(f"Network Error: {e.reason}")
except Exception as e:
    print(f"Unexpected error: {e}")
import json

try:
    # try to parse the data
    eu_data = json.loads(raw_json)
    print("Successfully parsed EU JSON data")
    print(eu_data)
except json.JSONDecodeError as e:
    print(f"JSON Error: {e}")
    eu_data = None
except Exception as e:
    print(f"Error parsing JSON: {e}")
countries = eu_data["countries"]
print(f"Found {len(countries)} countries:")
for country in countries:
    # make sure the JSON has the format we expect
    if "name" in country and "capital_city" in country:
        print(f"{country['name']} = {country['capital_city']}")
    else:
        print("Warning: Country missing name or capital_city field")
print("Population information:")
for country in countries:
    # make sure the JSON has the format we expect
    if "name" in country and "population" in country:
        print(f"{country['name']} = {country['population']}")
    else:
        print("Warning: Country missing population field")
import csv

try:
    with open("eu.csv", "w", encoding="utf-8", newline="") as fout:
```

```

        # specify the ordered list of fields
        fields = ["name", "population", "capital_city", "first_language"]
        writer = csv.DictWriter(fout, fieldnames=fields)
        # write the header row
        writer.writeheader()
        # write each country record
        for country in countries:
            writer.writerow(country)
        print(f"Successfully exported {len(countries)} countries to eu.csv")
except IOError as e:
    print(f"File Error: Could not write to eu.csv - {e}")
except csv.Error as e:
    print(f"CSV Error: {e}")
except Exception as e:
    print(f"Unexpected error: {e}")

```

What to change for a different dataset:

- Change the input file/URL (e.g., triathlon.csv, olympics2012.csv) to your dataset path.
- Update column names used in calculations (e.g., Gold/Silver/Bronze, Swim/Cycle/Run, Date/Close).
- If the dataset uses different missing-value markers, adjust cleaning (dropna vs fillna).
- If you group by a different category (e.g., Region instead of Province), update the groupby/crosstab columns.
- For plots, update the x/y columns and chart titles/labels.

Task 2

Task 2 - XML Parsing

Use Python to download a file containing a contact list in XML format from the URL:

<http://mlg.ucd.ie/modules/python/contacts.xml>

Parse the XML data that you have downloaded.

From the parsed XML data, extract the name, email address and phone number of each contact.

Store these contacts in a list of dictionaries, and print them out.

Export the contact data to a new file in CSV format.

Use Pickle to serialise the contact data to a file. Verify that you can deserialise the data again.

Solution code:

```

url = "http://mlg.ucd.ie/modules/python/contacts.xml"

try:
    response = urllib.request.urlopen(url)
    raw_xml = response.read().decode("utf-8")

```

```

        print("Successfully downloaded XML data")
        print(raw_xml)
except urllib.error.HTTPError as e:
    print(f"HTTP Error {e.code}: {e.reason}")
except urllib.error.URLError as e:
    print(f"Network Error: {e.reason}")
except Exception as e:
    print(f"Unexpected error: {e}")
import xml.etree.ElementTree

try:
    # parse the XML string to build a tree data structure
    tree = xml.etree.ElementTree.fromstring(raw_xml)
    print("Successfully parsed XML data")
except xml.etree.ElementTree.ParseError as e:
    print(f"XML Parse Error: {e}")
    tree = None
except Exception as e:
    print(f"Unexpected error parsing XML: {e}")
    tree = None
contacts = []
for entry in tree.findall("contact"):
    contact = {}

    # extract each field with error checking
    name_elem = entry.find("name")
    email_elem = entry.find("email")
    phone_elem = entry.find("phone")

    # check if all required elements exist
    if all(elem is not None and elem.text for elem in [name_elem, email_elem,
phone_elem]):
        contact["name"] = name_elem.text.strip()
        contact["email"] = email_elem.text.strip()
        contact["phone"] = phone_elem.text.strip()
        contacts.append(contact)
    else:
        print("Warning: Skipping contact with missing or empty fields")

print(f"Successfully parsed {len(contacts)} complete contacts")

# display contacts
print("Extracted contacts:")
for i, contact in enumerate(contacts, 1):
    print(f"{i}. {contact['name']}: \temail={contact['email']}
\tphone={contact['phone']}")
try:
    with open("contacts.csv", "w", encoding="utf-8", newline="") as fout:
        # specify the ordered list of fields
        fields = ["name", "email", "phone"]
        writer = csv.DictWriter(fout, fieldnames=fields)
        # write the header row
        writer.writeheader()
        # write each contact record
        for contact in contacts:

```

```

        writer.writerow(contact)
    print(f"Successfully exported {len(contacts)} contacts to contacts.csv")

except IOError as e:
    print(f"File Error: Could not write file - {e}")
except csv.Error as e:
    print(f"CSV Error: {e}")
except Exception as e:
    print(f"Unexpected error: {e}")
import pickle

try:
    # use Pickle to write the contact data out to a file
    with open("contacts.pkl", "wb") as fout:
        pickle.dump(contacts, fout)
    print(f"Successfully pickled {len(contacts)} contacts to contacts.pkl")

except IOError as e:
    print(f"File Error: Could not write file - {e}")
except Exception as e:
    print(f"Unexpected error: {e}")
# verify that pickling worked by reading it back in again
try:
    with open("contacts.pkl", "rb") as fin:
        backup = pickle.load(fin)
    print(f"Successfully loaded {len(backup)} contacts from pickle file")

    print("Loaded contacts:")
    # process all the contacts
    for i, contact in enumerate(backup, 1):
        # check we have all the required fields for each contact
        if all(key in contact for key in ["name", "email", "phone"]):
            print(f"{i}. {contact['name']}: \temail={contact['email']}
\tphone={contact['phone']}")
        else:
            print(f"{i}. Warning: Contact missing required fields")

except IOError as e:
    print(f"File Error: {e}")
except pickle.UnpicklingError as e:
    print(f"Pickle Error: {e}")
except Exception as e:
    print(f"Unexpected error: {e}")

```

What to change for a different dataset:

- Change the input file/URL (e.g., triathlon.csv, olympics2012.csv) to your dataset path.
- Update column names used in calculations (e.g., Gold/Silver/Bronze, Swim/Cycle/Run, Date/Close).
- If the dataset uses different missing-value markers, adjust cleaning (dropna vs fillna).
- If you group by a different category (e.g., Region instead of Province), update the groupby/crosstab columns.
- For plots, update the x/y columns and chart titles/labels.

Lab 05 Tasks - Solutions

Setup / Imports used in this lab:

```
import pandas as pd
```

Task 1

Task 1

Load the CSV file "olympics2012.csv" into a Pandas DataFrame, where each row is indexed by its ISO country code. Check the number of rows and the column names in the DataFrame.

Display the first 15 rows of the data.

Show the top 10 countries with the highest number of gold medals at the 2012 Olympics.

Solution code:

```
df = pd.read_csv("olympics2012.csv", index_col="ISO")
df.shape[0]
list(df.columns)
df.head(15)
# sort the Data Frame in descending order by gold medals
df_sorted = df.sort_values(by="Gold", ascending=False)
# show the top 10
df_sorted.head(10)
```

What to change for a different dataset:

- Change the input file/URL (e.g., triathlon.csv, olympics2012.csv) to your dataset path.
- Update column names used in calculations (e.g., Gold/Silver/Bronze, Swim/Cycle/Run, Date/Close).
- If the dataset uses different missing-value markers, adjust cleaning (dropna vs fillna).
- If you group by a different category (e.g., Region instead of Province), update the groupby/crosstab columns.
- For plots, update the x/y columns and chart titles/labels.

Task 2

Task 2

Create a new column in the DataFrame called "Total" which indicates the total number of medals won by each country. Show the top 10 countries with the highest number of total medals.

Display the subset of countries which:

1. Won 20 or more gold medals
2. Won 25 or more total medals
3. Won only bronze medals

Solution code:

```
# create the new column as the sum of the 3 medal types
df["Total"] = df["Gold"] + df["Silver"] + df["Bronze"]
# sort the Data Frame in descending order by total
df_sorted = df.sort_values(by="Total", ascending=False)
# show the top 10
df_sorted.head(10)
# filter for 20 or more gold medals
df.loc[df["Gold"]>=20]
# filter for 25 or more total medals
df.loc[df["Total"]>=25]
# filter for only bronze medals, no silver or gold
df.loc[(df["Gold"]==0) & (df["Silver"]==0) & (df["Bronze"]>0)]
```

What to change for a different dataset:

- Change the input file/URL (e.g., triathlon.csv, olympics2012.csv) to your dataset path.
- Update column names used in calculations (e.g., Gold/Silver/Bronze, Swim/Cycle/Run, Date/Close).
- If the dataset uses different missing-value markers, adjust cleaning (dropna vs fillna).
- If you group by a different category (e.g., Region instead of Province), update the groupby/crosstab columns.
- For plots, update the x/y columns and chart titles/labels.

Task 3

Task 3

Create a new column called "WeightedTotal", which computes a weighted total for the number of medals won by each country, with weights: Gold=3, Silver=2, Bronze=1.

Display the top 10 countries according to this score.

Solution code:

```
# create the new column as the weighted sum of the 3 medal types
df["WeightedTotal"] = 3*df["Gold"] + 2*df["Silver"] + df["Bronze"]
# sort the Data Frame in descending order by weighted total
df_sorted = df.sort_values(by="WeightedTotal", ascending=False)
# show the top 10
df_sorted.head(10)
```

What to change for a different dataset:

- Change the input file/URL (e.g., triathlon.csv, olympics2012.csv) to your dataset path.
- Update column names used in calculations (e.g., Gold/Silver/Bronze, Swim/Cycle/Run, Date/Close).
- If the dataset uses different missing-value markers, adjust cleaning (dropna vs fillna).
- If you group by a different category (e.g., Region instead of Province), update the groupby/crosstab columns.
- For plots, update the x/y columns and chart titles/labels.

Task 4

Task 4

Create a new column "TotalPerPop" which is calculated as the total number of medals won by a country per million population.

Display the top 20 countries according to this score.

Solution code:

```
# convert our population values to millions
normalised_pop = df["Population"]/1000000
# check the value for Ireland to make sure this is correct
normalised_pop["IRL"]
# now create the new column
df["TotalPerPop"] = df["Total"] / normalised_pop
# sort the Data Frame in descending order by gold medals
df_sorted = df.sort_values(by="TotalPerPop", ascending=False)
# show the top 20
df_sorted.head(20)
```

What to change for a different dataset:

- Change the input file/URL (e.g., triathlon.csv, olympics2012.csv) to your dataset path.
- Update column names used in calculations (e.g., Gold/Silver/Bronze, Swim/Cycle/Run, Date/Close).
- If the dataset uses different missing-value markers, adjust cleaning (dropna vs fillna).
- If you group by a different category (e.g., Region instead of Province), update the groupby/crosstab columns.
- For plots, update the x/y columns and chart titles/labels.

Lab 06 Tasks - Solutions

Setup / Imports used in this lab:

```
import pandas as pd
```

Task 1

Task 1 - Data Loading and Preparation

Use Python to download a file containing triathlon dataset in CSV format from the URL:

<http://mlg.ucd.ie/modules/python/triathlon.csv>

Load the dataset into a Pandas DataFrame, where the row index will be given by the athlete's bib number. Display the first 20 rows of the DataFrame.

The dataset might contain missing values. For instance, some athletes may have registered for the race for never actually. Other athletes might have started the race, but not completed all segments of the triathlon.

From the DataFrame, identify the number of missing values in each column. Then remove any rows which contain missing values (i.e. athletes who did not fully complete the race). How many rows are remaining?

Add a new column to the DataFrame, called *Finish*, which is the total time taken for the race for each athlete (i.e. Swim + T1 + Cycle + T2 + Run).

To verify the step above, sort the DataFrame, based on the *Finish* time, fastest to slowest. Display the top 10 fastest athletes overall:

Solution code:

```
# we could either download the file using urllib, or we can use Pandas to do
this for us
df = pd.read_csv("http://mlg.ucd.ie/modules/python/triathlon.csv",
index_col="Number")
# display first 20 rows
df.head(20)
# count number of missing values per column
df.isna().sum()
# number of rows currently
print("Data contains %d rows" % len(df))
# remove the rows with missing values
df = df.dropna()
# how many rows left?
print("Data now contains %d rows" % len(df))
# add the times for the different segments together
df["Finish"] = df["Swim"] + df["T1"] + df["Cycle"] + df["T2"] + df["Run"]
df.sort_values(by="Finish").head(10)
```

What to change for a different dataset:

- Change the input file/URL (e.g., triathlon.csv, olympics2012.csv) to your dataset path.
- Update column names used in calculations (e.g., Gold/Silver/Bronze, Swim/Cycle/Run, Date/Close).
- If the dataset uses different missing-value markers, adjust cleaning (dropna vs fillna).
- If you group by a different category (e.g., Region instead of Province), update the groupby/crosstab columns.
- For plots, update the x/y columns and chart titles/labels.

Task 2

Task 2 - Initial Data Analysis

What is the average finishing time for athletes? What is the slowest finishing time?

On average which segment of the race took the longest: swimming, cycling or running?

How many female and male athletes competed in the race? How many athletes from each Irish province competed in the race?

How many female and male athletes were from each of the 4 provinces?

(Hint: We can use a cross-tabulation)

Solution code:

```
# average time
print("Mean finish time = %.2f" % df["Finish"].mean())
# slowest time
print("Slowest finish time = %.2f" % df["Finish"].max())
# calculate the averages for each segment
mean_swim = df["Swim"].mean()
mean_cycle = df["Cycle"].mean()
mean_run = df["Run"].mean()
# check which took longest
if mean_swim > mean_cycle and mean_swim > mean_run:
    print("Swimming took longest")
elif mean_cycle > mean_swim and mean_cycle > mean_run:
    print("Cycling took longest")
else:
    print("Running took longest")
# create a frequency table for gender
df["Gender"].value_counts()
# create a frequency table for province
df["Province"].value_counts()
# apply a cross-tabulation on these two columns
pd.crosstab(df["Province"], df["Gender"])
```

What to change for a different dataset:

- Change the input file/URL (e.g., triathlon.csv, olympics2012.csv) to your dataset path.
- Update column names used in calculations (e.g., Gold/Silver/Bronze, Swim/Cycle/Run, Date/Close).
- If the dataset uses different missing-value markers, adjust cleaning (dropna vs fillna).
- If you group by a different category (e.g., Region instead of Province), update the groupby/crosstab columns.
- For plots, update the x/y columns and chart titles/labels.

Task 3

Task 3 - Further Data Analysis

Create a new column 'AgeCategory' that divides the runners into age categories: 16-19, 20-29, 30-39, 40-49, 50-65.

How many female and male athletes were from each age category?

(Hint: We can use a cross-tabulation and the 'AgeCategory' column from above)

What was finishing place for each athlete per province?

(Hint: We can perform group-by aggregation)

What were the average times for the three segments, per age category?

(Hint: We can perform group-by aggregation)

Solution code:

```
bins = [16, 20, 30, 40, 50, 65]
# here we specify right=False to exclude the rightmost edge
df["AgeCategory"] = pd.cut(df["Age"], bins=bins, right=False)
df.head()
# apply a cross-tabulation on the next two columns
pd.crosstab(df["AgeCategory"], df["Gender"])
# first aggregate by province
groups = df.groupby("Province", observed=False)
# just select the column we want
groups["Place"].mean()
# first aggregate by age category
groups = df.groupby("AgeCategory", observed=False)
# now calculate the means for the three segments
groups[["Swim", "Cycle", "Run"]].mean()
```

What to change for a different dataset:

- Change the input file/URL (e.g., triathlon.csv, olympics2012.csv) to your dataset path.
- Update column names used in calculations (e.g., Gold/Silver/Bronze, Swim/Cycle/Run, Date/Close).
- If the dataset uses different missing-value markers, adjust cleaning (dropna vs fillna).
- If you group by a different category (e.g., Region instead of Province), update the groupby/crosstab columns.
- For plots, update the x/y columns and chart titles/labels.

Lab 07

Setup / Imports used in this lab:

```
import pandas as pd
import matplotlib
import matplotlib.pyplot as plt
%matplotlib inline
```

Task 1

Task 1 - Data Loading

Use Python to download a file containing triathlon dataset in CSV format from the URL:

http://mlg.ucd.ie/modules/python/triathlon_clean.csv

Load the dataset into a Pandas DataFrame, where the row index will be given by the athlete's

bib number.

Display the first 10 rows of the DataFrame.

Solution code:

```
# we could either download the file using urllib, or we can use Pandas to do this for us
df = pd.read_csv("http://mlg.ucd.ie/modules/python/triathlon_clean.csv",
index_col="Number")
df.head(10)
```

What to change for a different dataset:

- Change the input file/URL (e.g., triathlon.csv, olympics2012.csv) to your dataset path.
- Update column names used in calculations (e.g., Gold/Silver/Bronze, Swim/Cycle/Run, Date/Close).
- If the dataset uses different missing-value markers, adjust cleaning (dropna vs fillna).
- If you group by a different category (e.g., Region instead of Province), update the groupby/crosstab columns.
- For plots, update the x/y columns and chart titles/labels.

Task 2

Task 2 - Data Visualisation

Use **bar charts** to visualise:

1. The number of athletes per age category
2. The number of athletes per province

(Hint: Start by calculating a frequency table for the relevant column)

Produce a **pie chart** showing the percentage of athletes by province

(Hint: Start by calculating a frequency table for the relevant column)

Produce a **histogram** visualisation of the distribution of athlete finish times:

Repeat the above, but this time produce a **histogram** visualisation of the distribution of finish times for female athletes only:

Produce three **scatter plots** which show:

1. The relationship between the time taken for the swimming and cycling segments.
2. The relationship between the time taken for the swimming and running segments.
3. The relationship between the time taken for the cycling and running segments.

Solution code:

```

# get the frequency counts
age_counts = df["AgeCategory"].value_counts()
# plot the frequencies
ax = age_counts.plot(kind="bar", fontsize=13, color="navy", figsize=(6, 4.5))
plt.ylabel("Number of Athletes", fontsize=13);
plt.show()
# get the frequency counts
province_counts = df["Province"].value_counts()
# plot the frequencies
ax = province_counts.plot(kind="bar", fontsize=13, color="teal", figsize=(6, 4.5))
plt.ylabel("Number of Athletes", fontsize=13);
plt.show()
# get the frequency counts
province_counts = df["Province"].value_counts()
# plot the frequencies
ax = province_counts.plot(kind="pie", figsize=(5, 5), fontsize=12,
autopct="%.1f%%",
                        colors=["royalblue", "red", "coral", "darkgreen"])
ax.set_ylabel(None);
plt.show()
ax = df["Finish"].hist(bins=10, figsize=(8, 4.5), color='green', grid=False,
rwidth=0.9)
plt.ylabel("Number of Athletes", fontsize=13)
plt.xlabel("Finish Time (seconds)", fontsize=13);
plt.show()
# filter based on gender first
df2 = df[df["Gender"]=="F"]
# then generate the histogram
ax = df2["Finish"].hist(bins=10, figsize=(8, 4.5), color='green', grid=False,
rwidth=0.9)
plt.ylabel("Number of Athletes", fontsize=13)
plt.xlabel("Finish Time (seconds)", fontsize=13);
plt.show()
ax = df.plot(kind="scatter", figsize=(8, 5), color='darkblue', s=40,
fontsize=13, x="Swim", y="Cycle")
plt.xlabel("Swimming Segment (seconds)", fontsize=13)
plt.ylabel("Cycling Segment (seconds)", fontsize=13);
plt.show()
ax = df.plot(kind="scatter", figsize=(8, 5), color='darkorange', s=40,
fontsize=13, x="Swim", y="Run")
plt.xlabel("Swimming Segment (seconds)", fontsize=13)
plt.ylabel("Running Segment (seconds)", fontsize=13);
plt.show()
ax = df.plot(kind="scatter", figsize=(8, 5), color='darkred', s=40,
fontsize=13, x="Cycle", y="Run")
plt.xlabel("Cycling Segment (seconds)", fontsize=13)
plt.ylabel("Running Segment (seconds)", fontsize=13);
plt.show()

```

What to change for a different dataset:

- Change the input file/URL (e.g., triathlon.csv, olympics2012.csv) to your dataset path.
- Update column names used in calculations (e.g., Gold/Silver/Bronze, Swim/Cycle/Run, Date/Close).

- If the dataset uses different missing-value markers, adjust cleaning (dropna vs fillna).
- If you group by a different category (e.g., Region instead of Province), update the groupby/crosstab columns.
- For plots, update the x/y columns and chart titles/labels.

Lab 08

Task 1

Task 1

Use Python to download the World Happiness Index data from the link below and load it into a Pandas DataFrame.

<http://mlg.ucd.ie/modules/python/happiness2024.csv>

Calculate basic summary statistics for the data and list the top 5 ranked countries for each measure.

Solution code:

```
import pandas as pd
# Pandas can download the data directly from the URL
df = pd.read_csv("http://mlg.ucd.ie/modules/python/happiness2024.csv",
index_col="country")
# check the basic structure
print(f"DataFrame has {df.shape[0]} rows, {df.shape[1]} columns")
print(f"Columns: {df.columns.tolist()}")
# display the first 10 rows
df.head(10)
# get basic statistics for the various measures (columns)
display(df.describe())

# list the top 5 ranked countries for each measure (column)
for column in df.columns:
    print(f"Top 5 countries for {column}:")
    display(df[column].sort_values(ascending=False).head(5).to_frame())
```

What to change for a different dataset:

- Change the input file/URL (e.g., triathlon.csv, olympics2012.csv) to your dataset path.
- Update column names used in calculations (e.g., Gold/Silver/Bronze, Swim/Cycle/Run, Date/Close).
- If the dataset uses different missing-value markers, adjust cleaning (dropna vs fillna).
- If you group by a different category (e.g., Region instead of Province), update the groupby/crosstab columns.
- For plots, update the x/y columns and chart titles/labels.

Task 2

Task 2

Calculate the correlations between the different measures in the data to help us understand which factors tend to co-occur.

Solution code:

```
# calculate the pairwise correlations
df_c = df.corr()
display(df_c)

# we can turn this into a sorted DataFrame to show ranking for column pairs
# with the highest and lowest correlation
from itertools import combinations
rows = []
for v1, v2 in combinations(df_c.columns, 2):
    rows.append({"Measure 1": v1, "Measure 2": v2, "Correlation": df_c[v1][v2]})
# show the ranked list
pd.DataFrame(rows).sort_values(by="Correlation", ascending=False)
```

What to change for a different dataset:

- Change the input file/URL (e.g., triathlon.csv, olympics2012.csv) to your dataset path.
- Update column names used in calculations (e.g., Gold/Silver/Bronze, Swim/Cycle/Run, Date/Close).
- If the dataset uses different missing-value markers, adjust cleaning (dropna vs fillna).
- If you group by a different category (e.g., Region instead of Province), update the groupby/crosstab columns.
- For plots, update the x/y columns and chart titles/labels.

Lab 11

Setup / Imports used in this lab:

```
from datetime import datetime
import pandas as pd
import matplotlib
import matplotlib.pyplot as plt
%matplotlib inline
pd.options.display.float_format = '{:,.2f}'.format
```

Task 1

Task 1

Load the stock price CSV files for both Meta (*stock-meta.csv*) and Amazon (*stock-amazon.csv*) into separate Pandas DataFrames.

(Note: Dates in these files are stored in "day/month/year" format)

Check the time range covered by the data for both stocks.

Solution code:

```
# load the first CSV file
df_meta = pd.read_csv("stock-meta.csv", index_col="date", parse_dates=True,
date_format="%d/%m/%Y")
print(f"Read {df_meta.shape[0]} observations for Meta")
display(df_meta.head())

# load the second CSV file
df_amazon = pd.read_csv("stock-amazon.csv", index_col="date", parse_dates=True,
date_format="%d/%m/%Y")
print(f"Read {df_amazon.shape[0]} observations for Amazon")
display(df_amazon.head())
start_meta, end_meta = df_meta.index.min(), df_meta.index.max()
print(f"Meta date range: {start_meta.date()} to {end_meta.date()}")

start_amazon, end_amazon = df_amazon.index.min(), df_amazon.index.max()
print(f"Amazon date range: {start_amazon.date()} to {end_amazon.date()}")
```

What to change for a different dataset:

- Change the input file/URL (e.g., triathlon.csv, olympics2012.csv) to your dataset path.
- Update column names used in calculations (e.g., Gold/Silver/Bronze, Swim/Cycle/Run, Date/Close).
- If the dataset uses different missing-value markers, adjust cleaning (dropna vs fillna).
- If you group by a different category (e.g., Region instead of Province), update the groupby/crosstab columns.
- For plots, update the x/y columns and chart titles/labels.

Task 2

Task 2

For each of the DataFrames, extract only the **close** stock prices for the year 2021. This should give you two new Pandas Series.

Solution code:

```
# extract 2021 close prices for Meta
meta_2021 = df_meta.loc["2021"]["close"]
print(f"Extracted {meta_2021.shape[0]} observations from 2021 for Meta")
display(meta_2021.head())

# extract 2021 close prices for Amazon
amazon_2021 = df_amazon.loc["2021"]["close"]
print(f"Extracted {amazon_2021.shape[0]} observations from 2021 for Amazon")
display(amazon_2021.head())
```

What to change for a different dataset:

- Change the input file/URL (e.g., triathlon.csv, olympics2012.csv) to your dataset path.
- Update column names used in calculations (e.g., Gold/Silver/Bronze, Swim/Cycle/Run, Date/Close).
- If the dataset uses different missing-value markers, adjust cleaning (dropna vs fillna).

- If you group by a different category (e.g., Region instead of Province), update the groupby/crosstab columns.
- For plots, update the x/y columns and chart titles/labels.

Task 3

Task 3

Create a new time series DataFrame with two columns, one for the Meta 2021 closing prices and one for the Amazon 2021 closing prices.

Solution code:

```
# create combined DataFrame with both stocks
df_combined = pd.DataFrame({
    "Meta": meta_2021,
    "Amazon": amazon_2021
})
df_combined.head()
```

What to change for a different dataset:

- Change the input file/URL (e.g., triathlon.csv, olympics2012.csv) to your dataset path.
- Update column names used in calculations (e.g., Gold/Silver/Bronze, Swim/Cycle/Run, Date/Close).
- If the dataset uses different missing-value markers, adjust cleaning (dropna vs fillna).
- If you group by a different category (e.g., Region instead of Province), update the groupby/crosstab columns.
- For plots, update the x/y columns and chart titles/labels.

Task 4

Task 4

Using the data from Task 3, plot the closing prices for Meta and Amazon, using a 7-day rolling mean.

Solution code:

```
# apply the rolling mean
df_rm = df_combined.rolling(7).mean()
# create the plot
p = df_rm.plot(figsize=(11, 5.5), fontsize=12, zorder=3)
plt.xlabel("Date", fontsize=12)
plt.title("7-Day Rolling Mean of Stock Prices", fontsize=12)
plt.ylabel("Close Price ($)", fontsize=12)
plt.grid(axis="y")
plt.show()
```

What to change for a different dataset:

- Change the input file/URL (e.g., triathlon.csv, olympics2012.csv) to your dataset path.

- Update column names used in calculations (e.g., Gold/Silver/Bronze, Swim/Cycle/Run, Date/Close).
- If the dataset uses different missing-value markers, adjust cleaning (dropna vs fillna).
- If you group by a different category (e.g., Region instead of Province), update the groupby/crosstab columns.
- For plots, update the x/y columns and chart titles/labels.

Task 5

Task 5

Resample the DataFrame from Task 3 to monthly time windows, and plot the new monthly time series.

Solution code:

```
# convert from day frequency to month (ME) frequency, by averaging the values
df_monthly = df_combined.resample("ME").mean()

# create the plot
p = df_monthly.plot(figsize=(11, 5.5), fontsize=12, lw=2, zorder=3)
plt.title("Monthly Average Stock Prices", fontsize=12)
plt.ylabel("Closing Stock Price", fontsize=12)
plt.xlabel("Month", fontsize=12)
plt.ylabel("Close Price ($)", fontsize=12)
plt.grid(axis="y")
plt.show()
```

What to change for a different dataset:

- Change the input file/URL (e.g., triathlon.csv, olympics2012.csv) to your dataset path.
- Update column names used in calculations (e.g., Gold/Silver/Bronze, Swim/Cycle/Run, Date/Close).
- If the dataset uses different missing-value markers, adjust cleaning (dropna vs fillna).
- If you group by a different category (e.g., Region instead of Province), update the groupby/crosstab columns.
- For plots, update the x/y columns and chart titles/labels.

Task 2 - Classification (Train/Test + Cross-Validation)

This section contains lab questions and solutions for classification using train/test evaluation and cross-validation.

Lab 09 Tasks - Solutions

Setup / Imports used in this lab:

```
import pandas as pd
from sklearn.model_selection import cross_val_score
# imports for plotting
import matplotlib
import matplotlib.pyplot as plt
%matplotlib inline
```

Task 1

Task 1

Load the dataset from the file `*music.csv*` and examine the number of songs having each target label.

Remove any non-numeric features from the dataset, and then separate out the features to use for classification from the target label information.

Solution code:

```
df = pd.read_csv("music.csv")
df.head()
# check the number with each class label
df["genre"].value_counts()
# remove the non-numeric features
df.drop(["artist_name", "track_name"], axis=1, inplace=True)
df.head()
# the target label
target = df["genre"]
# remove the target label, giving us just the numeric features for
classification
data = df.drop("genre", axis=1).copy()
```

What to change for a different dataset:

- Change TARGET / label column name to your dataset's target.
- Update feature selection (numeric-only, or include categorical/text preprocessing).
- Change the train/test split ratio and random_state if the question specifies.
- Adjust the model hyperparameters (e.g., k in KNN) and metric (accuracy vs macro-F1) to match the question.
- If classes are imbalanced, prefer stratified split and macro-averaged metrics.

Task 2

Task 2

Generate a 60/40 random training and test split of the data. Based on this split, evaluate the accuracy and F1-score achieved by a KNN classifier (for **k=1** neighbour).

Use a **confusion matrix** to illustrate where the errors lie with the classifier above.

Solution code:

```
from sklearn.model_selection import train_test_split
# use 60% for training, 40% for testing
data_train, data_test, target_train, target_test = train_test_split(data,
    target,
    test_size=0.4, random_state=1)
print(f"Training set has {data_train.shape[0]} examples")
print(f"Test set has {data_test.shape[0]} examples")
# train the classifier
from sklearn.neighbors import KNeighborsClassifier
knn = KNeighborsClassifier(n_neighbors=1)
m = knn.fit(data_train, target_train)
# make predictions for the test set
predicted = knn.predict(data_test)
# evaluate accuracy
from sklearn.metrics import accuracy_score
acc = accuracy_score(target_test, predicted)
print(f"Accuracy = {acc:.3f}")
# evaluate F1 score
from sklearn.metrics import f1_score
print(f"F1(Rock) = {f1_score(target_test, predicted, pos_label='Rock'):.3f}")
print(f"F1(Pop) = {f1_score(target_test, predicted, pos_label='Pop'):.3f}")
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
# build the confusion matrix
cm = confusion_matrix(target_test, predicted)
# display it visually
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=knn.classes_)
disp.plot(cmap="Oranges")
plt.show()
```

What to change for a different dataset:

- Change TARGET / label column name to your dataset's target.
- Update feature selection (numeric-only, or include categorical/text preprocessing).
- Change the train/test split ratio and random_state if the question specifies.
- Adjust the model hyperparameters (e.g., k in KNN) and metric (accuracy vs macro-F1) to match the question.
- If classes are imbalanced, prefer stratified split and macro-averaged metrics.

Task 3

Task 3

Use 5-fold cross-validation to evaluate the accuracy achieved by a KNN (**k=1**) classifier on the data.

Repeat the process above for different parameter values of **k**, from 2 to 10 neighbours.

Generate a plot of the different accuracy values achieved for different values of *k*. Which value of *k* yields the highest accuracy?

Solution code:

```
knn = KNeighborsClassifier(n_neighbors=1)
fold_scores = cross_val_score(knn, data, target, cv=5, scoring="accuracy")
mean_accuracies = {1: fold_scores.mean()}
print(f"KNN (k=1): Mean cross-validation accuracy = {mean_accuracies[1]:.3f}")
for k in range(2, 11):
    # apply KNN for the specified value
    knn = KNeighborsClassifier(n_neighbors=k)
    fold_scores = cross_val_score(knn, data, target, cv=5, scoring="accuracy")
    mean_accuracies[k] = fold_scores.mean()
    print(f"KNN (k={k}): Mean cross-validation accuracy = {mean_accuracies[k]:.3f}")
# generate the plot
df_mean_accuracies = pd.Series(mean_accuracies)
ax = df_mean_accuracies.plot(figsize=(8.5, 5), fontsize=12, lw=2, marker='o',
color="darkorange")
ax.set_ylabel("Cross-validation Accuracy", fontsize=12)
ax.set_xlabel("Number of Neighbours (k)", fontsize=12)
ax.yaxis.grid()
ax.set_xlim(1, 10)
plt.show()
# sort the values to see the parameter giving the highest accuracy
df_sorted = df_mean_accuracies.sort_values(ascending=False)
print(df_sorted)
# get the value of k which gives the highest accuracy
best_k = df_sorted.index[0]
print(f"Best value is k={best_k}")
```

What to change for a different dataset:

- Change TARGET / label column name to your dataset's target.
- Update feature selection (numeric-only, or include categorical/text preprocessing).
- Change the train/test split ratio and random_state if the question specifies.
- Adjust the model hyperparameters (e.g., k in KNN) and metric (accuracy vs macro-F1) to match the question.
- If classes are imbalanced, prefer stratified split and macro-averaged metrics.

Lab 10 Tasks - Solutions

Task 1

Task 1 - Preprocessing

Read the Twitter dataset from the CSV file 'tweet-language.tsv' into a Pandas DataFrame, where the row index is given by 'Tweet Id'.

Our target label for classification here is going to be the column 'English' -- a value of 1 indicates that a tweet is in English, while a value of 0 indicates it is written in another language.

From this column, check the number of tweets in the dataset for each class.

Using the DataFrame and functionality from scikit-learn, create a vector representations of the documents. For real applications we would want to use a custom tokenizer to handle the specifics of tweets (e.g. mentions, hashtags etc). However, for this example we can just use the standard scikit-learn tokenizer and a simple `*CountVectorizer*`.

Note that we should not use any "stop words" here. For language detection, common stop words might actually prove to be useful features.

Solution code:

```
import pandas as pd
df = pd.read_csv("tweet-language.tsv", sep="\t").set_index("Tweet ID")
print("Read %d documents" % len(df))
df.head(10)
target = df["English"]
target.value_counts()
# the content for all documents
documents = df["Tweet"]
# apply the vectorization process
from sklearn.feature_extraction.text import CountVectorizer
vectorizer = CountVectorizer(min_df = 10, stop_words=None)
X = vectorizer.fit_transform(documents)
# check the size of the resulting representation
print(X.shape)
# check the number of terms/words in our preprocessed vocabulary
terms = vectorizer.get_feature_names_out()
print("Vocabulary has %d distinct terms" % len(terms))
```

What to change for a different dataset:

- Change TARGET / label column name to your dataset's target.
- Update feature selection (numeric-only, or include categorical/text preprocessing).
- Change the train/test split ratio and random_state if the question specifies.
- Adjust the model hyperparameters (e.g., k in KNN) and metric (accuracy vs macro-F1) to match the question.
- If classes are imbalanced, prefer stratified split and macro-averaged metrics.

Task 2

Task 2 - Classification and Train/Test Evaluation

Train a kNN classification model with 3 neighbours, and evaluate the accuracy of this model using a single train/test split, so that we have 70% of the tweets in the training set and 30% in the test set.

Repeat the classification and evaluation process again using a different train/test split. Did the classifier achieve the same accuracy score as before?

Solution code:

```
# perform the split - note test_size=0.3 means 30% assigned to the test set
from sklearn.model_selection import train_test_split
data_train, data_test, target_train, target_test = train_test_split(X, target,
test_size=0.3)
# we will just check how many tweets in each set
print("Training set has %d tweets" % data_train.shape[0] )
print("Test set has %d tweets" % data_test.shape[0] )
# prepare the k-NN classification model, for 3 nearest neighbours in this case
from sklearn.neighbors import KNeighborsClassifier
model = KNeighborsClassifier(n_neighbors=3)
model.fit(data_train, target_train)
# make predictions for the tweets in the test set
predicted = model.predict(data_test)
predicted
# now we will evaluate the performance of the classifier
from sklearn.metrics import accuracy_score
print("Accuracy = %.4f" % accuracy_score(target_test, predicted))
data_train, data_test, target_train, target_test = train_test_split(X, target,
test_size=0.3)
model = KNeighborsClassifier(n_neighbors=3)
model.fit(data_train, target_train)
predicted = model.predict(data_test)
print("Accuracy = %.4f" % accuracy_score(target_test, predicted))
```

What to change for a different dataset:

- Change TARGET / label column name to your dataset's target.
- Update feature selection (numeric-only, or include categorical/text preprocessing).
- Change the train/test split ratio and random_state if the question specifies.
- Adjust the model hyperparameters (e.g., k in KNN) and metric (accuracy vs macro-F1) to match the question.
- If classes are imbalanced, prefer stratified split and macro-averaged metrics.

Task 3

Task 3 - Classification and Cross-Validation

If we re-run the evaluation above several times, we will get different performance scores depending on the randomly-generated training/test split that we are using. A more robust strategy involves using *k-fold cross-validation* to evaluate a classifier.

Evaluate the kNN classifier from above, but this time using 5-fold cross validation. The model in each fold should be evaluated using accuracy. Calculate the overall average accuracy across all 5 folds.

Solution code:

```
from sklearn.model_selection import cross_val_score
# create a single classifier
model = KNeighborsClassifier(n_neighbors=3)
# apply 5-fold cross-validation, measuring accuracy each time
```

```
acc_scores = cross_val_score(model, X, target, cv=5, scoring="accuracy")
# represent the results as a Pandas Series
labels = ["Fold %d" % i for i in range(1, len(acc_scores)+1)]
s_acc = pd.Series(acc_scores, index = labels)
s_acc
# overall average accuracy
print("Mean accuracy: %.4f" % s_acc.mean())
```

What to change for a different dataset:

- Change TARGET / label column name to your dataset's target.
- Update feature selection (numeric-only, or include categorical/text preprocessing).
- Change the train/test split ratio and random_state if the question specifies.
- Adjust the model hyperparameters (e.g., k in KNN) and metric (accuracy vs macro-F1) to match the question.
- If classes are imbalanced, prefer stratified split and macro-averaged metrics.

Task 3 - Predictive Modelling, Regression, Feature Analysis

This section contains lab questions and solutions for regression and feature-based analysis (including feature subset evaluation).

Lab 08

Task 3

Task 3

Apply **simple linear regression** to model the relationship where ***gdp*** is the input variable (predictor) and ***health*** is the target variable (response).

Create a scatter plot with the regression line overlaid to visualise the relationship.

Solution code:

```
from sklearn.linear_model import LinearRegression
import matplotlib.pyplot as plt
%matplotlib inline

# prepare data for regression analysis
x = df[["gdp"]].values
y = df[["health"]].values

# build and fit the regression model
model = LinearRegression()
model.fit(x, y)
print(f"Coefficient: {model.coef_[0][0]:.4f}")
print(f"Intercept: {model.intercept_[0]:.4f}")

# visualise the relationship with regression line
plt.figure(figsize=(8, 5.5))
plt.scatter(x, y, alpha=0.7)

# plot the fitted regression line
m = model.coef_[0]
b = model.intercept_
x_min = x.min()
x_max = x.max()
# this draws the regression line from x_min to x_max using the fitted slope and intercept
plt.plot([x_min, x_max], [m * x_min + b, m * x_max + b], "r", linewidth=2)

plt.xlabel("GDP per Capita", fontsize=12)
plt.ylabel("Health (Life Expectancy)", fontsize=12)
plt.title("GDP vs Health: Linear Regression")
plt.show()
```

What to change for a different dataset:

- Change the target and feature columns (e.g., gdp/health) to the variables requested in the question.

- If comparing multiple models/relationships, repeat the same code with different X/Y columns.
- For feature-subset experiments, update the list of feature sets to match the dataset columns.
- If the task asks for interpretation, report coefficients/correlations/importances with short written conclusions.

Task 4

Task 4

Repeat the **simple linear regression** process from Task 4, but this time use **generosity** as the target (response) variable.

Visually compare the strength of the relationships between the two regression models from Tasks 4 and 5.

Solution code:

```
# analyse gdp vs generosity relationship
x = df[["gdp"]].values
y = df[["generosity"]].values

# build and fit the regression model
model = LinearRegression()
model.fit(x, y)
print(f"Coefficient: {model.coef_[0][0]:.4f}")
print(f"Intercept: {model.intercept_[0]:.4f}")

# visualise the relationship
plt.figure(figsize=(8, 5.5))
plt.scatter(x, y, alpha=0.7)

# plot the fitted regression line
m = model.coef_[0]
b = model.intercept_
x_min = x.min()
x_max = x.max()
# this draws the regression line from x_min to x_max using the fitted slope and intercept
plt.plot([x_min, x_max], [m * x_min + b, m * x_max + b], "r", linewidth=2)

plt.xlabel("GDP per Capita", fontsize=12)
plt.ylabel("Generosity", fontsize=12)
plt.title("GDP vs Generosity: Linear Regression", fontsize=12)
plt.show()
```

What to change for a different dataset:

- Change the target and feature columns (e.g., gdp/health) to the variables requested in the question.
- If comparing multiple models/relationships, repeat the same code with different X/Y columns.
- For feature-subset experiments, update the list of feature sets to match the dataset columns.

- If the task asks for interpretation, report coefficients/correlations/importances with short written conclusions.

Task 5

Task 5

We will now use **multiple linear regression** to model the relationship between several socioeconomic measures and health outcomes.

Firstly, select ***gdp***, ***social_support***, ***freedom***, and ***corruption*** as the input (predictor) variables, with ***health*** as the target variable.

Next, apply **Z-score normalisation** to the input data. This allows the direct comparison of coefficient magnitudes.

Finally, build and fit a multiple linear regression model. Compare the resulting model's coefficients to determine which factor has the strongest relationship with health.

Hint: We can easily apply Z-score normalisation with scikit-learn using ``sklearn.preprocessing.StandardScaler``

Solution code:

```
from sklearn.preprocessing import StandardScaler

# prepare the data for multiple regression
predictors = ["gdp", "social_support", "freedom", "corruption"]
X = df[predictors].values
y = df[["health"]].values

# apply normalisation to allow direct comparison of coefficients
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# build and fit the multiple regression model on the normalised data
model = LinearRegression()
model.fit(X_scaled, y)

# report the coefficients for each predictor
# note if the features in the data are strongly correlated, coefficients can be
# unreliable
# and may not show true feature importance
print("Coefficients:")
for predictor, coef in zip(predictors, model.coef_[0]):
    print(f"- {predictor}: {coef:.4f}")
```

What to change for a different dataset:

- Change the target and feature columns (e.g., gdp/health) to the variables requested in the question.

- If comparing multiple models/relationships, repeat the same code with different X/Y columns.
- For feature-subset experiments, update the list of feature sets to match the dataset columns.
- If the task asks for interpretation, report coefficients/correlations/importances with short written conclusions.

Lab 09 Tasks - Solutions

Setup / Imports used in this lab:

```
import pandas as pd
from sklearn.model_selection import cross_val_score
# imports for plotting
import matplotlib
import matplotlib.pyplot as plt
%matplotlib inline
```

Task 4

Task 4

For certain datasets, classification may work better on a subset of features, rather than on the entire feature set (e.g. when noisy or misleading features are removed).

Using the KNN and the best value of *k* identified in Task 3, compare classification performance for the three feature subsets in the lists below. Which subset gives the highest accuracy?

Solution code:

```
feature_subsets = {
    "rhythm_mood": ["danceability", "energy", "tempo", "valence"],
    "acoustic_vocal": ["acousticness", "instrumentalness", "liveness",
    "speechiness"],
    "energy_loudness": ["energy", "tempo", "valence", "loudness"]
}
# evaluate each subset using KNN and 5-fold cross-validation
for name, features in feature_subsets.items():
    data_subset = data[features]
    knn = KNeighborsClassifier(n_neighbors=best_k)
    fold_scores = cross_val_score(knn, data_subset, target, cv=5,
    scoring="accuracy")
    mean_acc = fold_scores.mean()
    print(f"{name} feature subset - KNN (k={best_k}): Mean cross-validation
    accuracy = {mean_acc:.3f}")
```

What to change for a different dataset:

- Change the target and feature columns (e.g., gdp/health) to the variables requested in the question.
- If comparing multiple models/relationships, repeat the same code with different X/Y columns.
- For feature-subset experiments, update the list of feature sets to match the dataset columns.

- If the task asks for interpretation, report coefficients/correlations/importances with short written conclusions.